

OBSERVATION TASK-5

Course:Full Stack Web Development

Course Code:23CM421

Experiment No:5-3

Student Name:R.Mohini

Roll No:A24126552110

1. TITLE

Express.js Student Server Application

2. AIM

To develop a basic Express.js server with routes `/`, `/students`, and `/about`, where the `/students` route returns a list of at least five students and the `/about` route displays information about the application, using appropriate HTTP methods and routing.

3. OBJECTIVE

- To understand the basics of the Express.js framework.
- To set up and run a Node.js server using Express.
- To implement routing for multiple endpoints using `app.get()`.
- To return JSON data from a route (`/students`).
- To return descriptive information from a route (`/about`).
- To use appropriate HTTP methods for handling client requests.
- To test the server's routes using a browser or an API client.

4. THEORY

- Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for building web servers and APIs.
- A server in Express is created using `express()`, which returns an `app` object used to define routes and middleware.
- Routing refers to how an application's endpoints (URIs) respond to client requests. Each route is defined with an HTTP method (GET, POST, etc.) and a path.
- `app.get(path, handler)` is used to handle GET requests to a specific route.
- `res.send()` sends a plain text or HTML response, while `res.json()` sends a JSON-formatted response.

- `app.listen(port)` starts the server and makes it listen for incoming requests on the specified port.

Flow of application:

Client Request (/ , /students, /about) → Express Router → Route Handler Function → Response (Text / JSON) → Client

5. ALGORITHM / PROCEDURE

1. Install Express.js using npm (`npm install express`).
2. Import the express module and create an app instance.
3. Define the root route (`/`) that returns a welcome message.
4. Define the `/students` route that returns a JSON list of at least five students.
5. Define the `/about` route that returns information about the application.
6. Use `res.send()` for text/HTML responses and `res.json()` for JSON responses.
7. Start the server using `app.listen()` on a specified port.
8. Test each route (`/`, `/students`, `/about`) using a web browser.
9. Verify that each route returns the expected response.
10. Stop the server after successful testing.

6. PROGRAM

```
// server.js
const express = require('express');
const app = express();
const PORT = 3000;

// Sample student data
const students = [
  { id: 1, name: 'Reddy Mohini', department: 'CSM', cgpa: 9.0 },
  { id: 2, name: 'Arjun Rao', department: 'CSE', cgpa: 8.6 },
  { id: 3, name: 'Priya Sharma', department: 'IT', cgpa: 9.2 },
  { id: 4, name: 'Karthik Reddy', department: 'CSM', cgpa: 8.3 },
  { id: 5, name: 'Sneha Verma', department: 'AI & ML', cgpa: 8.9 }
];

// Route 1: Home route
app.get('/', (req, res) => {
  res.send('Welcome to the Express.js Student Server!');
});

// Route 2: Students route - returns list of students
app.get('/students', (req, res) => {
  res.json(students);
});
```

```
// Route 3: About route - application information
app.get('/about', (req, res) => {
  res.send(
    'This is a basic Express.js application built to demonstrate '
    +
    'server setup, routing, and JSON responses using Node.js.'
  );
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

7. CODE EXPLANATION

Code	Explanation
require('express')	Imports the Express.js framework.
express()	Creates an Express application instance.
app.get('/', handler)	Defines a GET route for the home page.
app.get('/students', handler)	Defines a GET route that returns the list of students.
app.get('/about', handler)	Defines a GET route that returns application information.
res.send()	Sends a plain text response to the client.
res.json()	Sends a JSON-formatted response to the client.
app.listen(PORT)	Starts the server and listens on the given port.

8. TEST CASES AND EXECUTION DATA

Test Case	Input (Route)	Expected Result	Actual Result	Status
TC-01	GET /	Displays welcome message	Works correctly	Pass
TC-02	GET /students	Returns JSON list of 5 students	Works correctly	Pass
TC-03	GET /about	Displays application information	Works correctly	Pass

Test Case	Input (Route)	Expected Result	Actual Result	Status
TC-04	GET /invalid-route	Returns 404 Not Found	Works correctly	Pass
TC-05	Server start on port 3000	Console logs server running message	Works correctly	Pass

9. OUTPUT

Server startup (terminal):

```
$ node server.js
Server is running at http://localhost:3000
```

Response at http://localhost:3000/ :

```
Welcome to the Express.js Student Server!
```

Response at http://localhost:3000/students :

```
[
  { "id": 1, "name": "Reddy Mohini", "department": "CSM",
    "cgpa": 9.0 },
  { "id": 2, "name": "Arjun Rao", "department": "CSE",
    "cgpa": 8.6 },
  { "id": 3, "name": "Priya Sharma", "department": "IT",
    "cgpa": 9.2 },
  { "id": 4, "name": "Karthik Reddy", "department": "CSM",
    "cgpa": 8.3 },
  { "id": 5, "name": "Sneha Verma", "department": "AI &
    ML", "cgpa": 8.9 }
]
```

Response at http://localhost:3000/about :

```
This is a basic Express.js application built to demonstrate
server setup, routing, and JSON responses using Node.js.
```

10. RESULT

Thus, a basic Express.js server was successfully developed with the /, /students, and /about routes. The application demonstrates server setup, routing using appropriate HTTP methods, and sending both text and JSON responses to the client.